



**Utrecht
University**

Editing the editor: proposing an extensible language extension for Agda

Author:
Job Vonk

First Supervisor:
Dr. W. S. Swierstra

Second Supervisor:
I. G. de Wolff MSc.

Daily Supervisor:
L. B. Chonavel MSc.

August, 2026

Table of Contents

1. Introduction	1
2. Background	3
2.1. Visual Studio Code	3
2.2. Existing Agda extensions	4
2.2.1. Implementations	5
2.3. Meta-programming	7
2.3.1. In Idris1	7
2.3.2. In Agda	8
2.3.3. In Lean4	8
3. Building Avea	10
3.1. Running a Visual Studio Code extension	10
3.2. Sending interactions	11
3.3. Receiving and handling responses	12
3.4. Reacting to changes from the user	14
3.5. Input mode	15
4. Challenges with programming in Agda	16
4.1. Importing Javascript libraries	16
4.2. Handling asynchronicity	17
4.3. Mitigating stack overflows	18
4.4. Evaluation	20
5. Adding extensibility	21
5.1. Start of the Avea extension EDSL	23
5.2. Edit-time actions in Agda	24
5.3. Changing Agda's meta-language	27
5.4. Port of Lean4's user widgets to Avea and Agda	28
6. Discussion	30
7. Future work	31
8. Bibliography	32

1. Introduction

Dependently-typed programming languages provide a means to prove that software meets its specification. This gives rise to software that is completely verified; that is, it is proven that a library or a program behaves in accordance to its defined specification. Dependent types can be much more precise about the programs they describe. The type can encode invariants that must hold during the function’s execution, or preconditions on the parameters.

Even though complete verification sounds ideal, there is one major flaw: using formal methods, like dependently-typed languages and theorem provers, to verify code can be labour intensive. It takes mathematically skilled developers, and quite some time (compared to informal quality assurance methods like testing) to fully verify a system [1]. Good tooling in dependently-typed languages can lift some of the burden of verifying a system: many dependently-typed languages need interactive development environments (IDEs) to facilitate the incremental construction of correct software.

Additionally, dependently-typed languages are also often used as theorem provers. However without tooling, such a system can be hard to prove theorems in, especially since normalisation of the system can be hard to reason about without querying the system. Additionally, these systems benefit even more from an IDE – for theorem provers often called an *interactive theorem prover* (ITP) – since proofs are written as an interactive dialogue between the theorem prover and the developers.

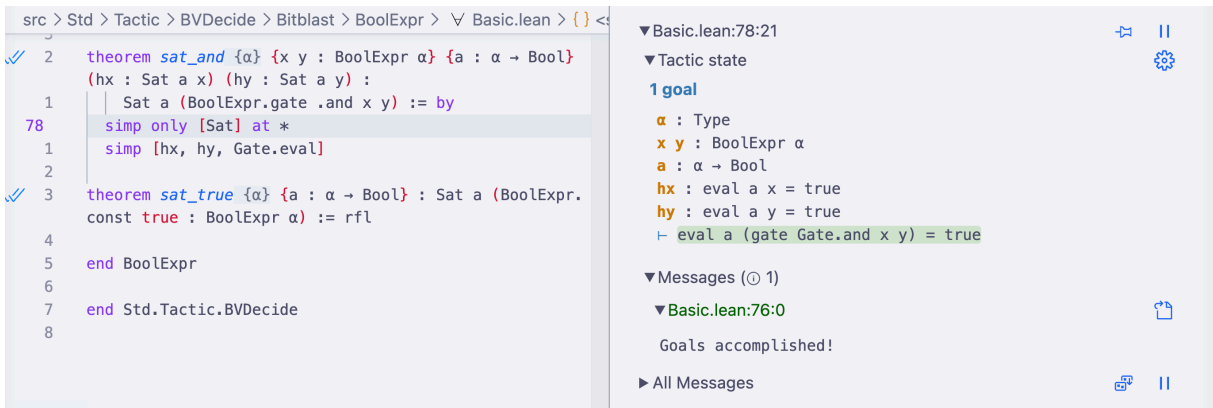


Figure 1: Example of using Lean4, an interactive theorem prover, to prove a theorem. The panel on the right shows information about the current state of the proof and what remains to be done at the location of the cursor.

Juhošová et al. [2] have investigated what obstacles students experience when learning to use Agda as an interactive theorem prover. Their advice is to improve robustness and accessibility, where one point they mention for both improvement areas is polishing and improving the existing tooling.

Juhošová et al. [3] also conducted a study about type driven development. The participants of the study were asked to suggest improvements to languages that offer type-driven development and their tooling. Some of the improvements the participants suggested were “wider editor support for interactive modes”, “better IDE support for large-scale proof engineering” and more support for refactoring. Backx [4] has also investigated how (novice) dependently-typed language programmers write code. He, too, saw that users experience a lack of tooling support for refactorings, and also recommends authors of dependently-typed interactive theorem provers to improve the editor tooling with, for example, a search feature for lemmas, or refactoring features that can also be found in non-dependently typed languages.

In Agda we see little innovation in editor features, even though there is strong incentive from users to develop new features. All of the current extensions for Agda support roughly the same feature

set. These editor features are provided by the compiler, and the extensions all merely implement an interface to these compiler features. One reason for the lack of innovation might be that extending this feature set is quite difficult: the features are implemented within the Agda compiler, which is a large Haskell code base, which is hard to extend. Additionally, these features are quite tightly coupled to Agda's elaborator and unification system, which are complex systems that are hard to interact with without much prior experience.

Another reason for the little innovation might be the fact that none of the extensions are written in Agda itself, rather they are implemented in various languages like ReScript, Haskell or Emacs Lisp. This raises the barrier of entry for people familiar with Agda wanting to contribute a new extension feature.

These two observations motivate the following research questions:

1. Is Agda a suitable language to write practical programs with?

Current big Agda projects are libraries like the standard library, `unimath`¹ and `cubical`², which highlight Agda's strength as a theorem prover. However, at the Agda wiki's home page, the language is introduced as a dependently-typed *programming language* before being introduced as a proof assistant³. Strangely, there does not seem to be a big practical programming project showing Agda's viability as a programming language. By writing a language extension in Agda, we aim to assess whether or not Agda is a viable programming language as opposed to a theorem prover. After the development, we can reflect on issues in the language or its tooling, and report a list of improvements to make Agda more suitable for practical applications.

2. Is it possible to make an extensible Agda extension written in Agda itself?

We will answer this question by writing an Agda language extension for Visual Studio Code in Agda – called Avea. The goal is to explore whether we can leverage Agda's meta-programming capabilities to allow developers to write their own sophisticated tooling in Agda itself. This process of extending Agda's tooling should be easier than the current process of extending the Agda compiler, and updating every extension to use the new tooling feature.

¹<https://unimath.github.io/agda-unimath/>

²<https://github.com/agda/cubical>

³<https://agda.readthedocs.io/en/latest/getting-started/what-is-agda.html>

2. Background

Before discussing Avea's implementation,

- we need to familiarise ourselves with the available features of Visual Studio Code, the editor Avea will operate in.
- Then we will discuss features in existing Agda extensions, and how they interact with the Agda compiler.
- Lastly, we will discuss metaprogramming in several dependently-typed languages, which is foundational for Avea's extensibility.

2.1. Visual Studio Code

Visual Studio Code (VS Code) is an open-source code editor developed by Microsoft. According to last year's StackOverflow Developer survey, 79.6% of the respondents used Visual Studio Code regularly in 2025 [5]. While the editor ships with support for Javascript/Typescript support out of the box, it has a robust extension system and a marketplace with extensions for many other programming languages. This includes extensions for mainstream languages like C++, C#, Rust, etc. However, Visual Studio Code has also been target for dependently-typed languages. For example, Rocq and Lean4 have sophisticated language extensions.



Figure 2: Screenshot of Visual Studio Code, the numbers correspond to the enumeration in the paragraph below.

Extensions can be written in Javascript or Typescript. When VS Code executes the code for an extension, it provides an API which allows extensions to interface with the different parts of the editor. The components relevant to Avea's development are highlighted in Figure 2 and discussed below.

1. The API allows extension developers to create panels, these are implemented as an HTML iframe. This means that the panel can display arbitrary HTML, CSS and Javascript. Additionally, the extension can send and receive messages to and from the panel using the window's `postMessage` method and the window's `message` event. This panel is used by the `lean4-vscode` extension as the

info view, where information about the proof state is displayed (as seen in Figure 1), in Agda this window is called the display buffer.

2. The editor itself can also be interacted with. Extension developers can request parts of the buffer based on positions within the document. Or the buffer can be programmatically edited, or decorated using syntax highlighting.
3. Extensions can also log information to VS Code’s output window. The API for this looks like a simple logging library, with different levels of logs. The logs can be viewed in the output window per extension, and can also be exported from there. This is especially useful during debugging.
4. There is a status bar at the bottom of the screen that displays small amounts of information – typically single words or icons. Commonly, the status bar contains the the location of the cursor, how many errors the project contains or the language of the currently active buffer.

2.2. Existing Agda extensions

The Agda ecosystem has a number of extensions for different programming editors. All of these extensions behave very similarly and feature a similar set of tools:

- Agda has many different **commands** that users can invoke using keybindings. These commands can be categorised in roughly three categories.

Goal refinement commands allow the user to enter (parts of) expressions into a goal and Agda will type-check or refine them based on the goal’s type. Examples of commands in this category are:

- the refine command, which inserts question marks for every missing argument of the given expression;
- the make case command, which creates exhaustive pattern matching clauses for a parameter, where possible (see Figure 3).

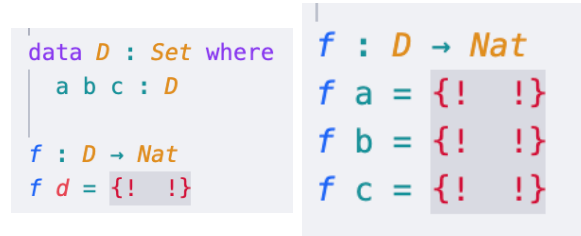


Figure 3: Example use of the make case command: in a goal the user can type C-c C-c in a goal (left image), and specify which parameters to case split on and all clauses appear (right image).

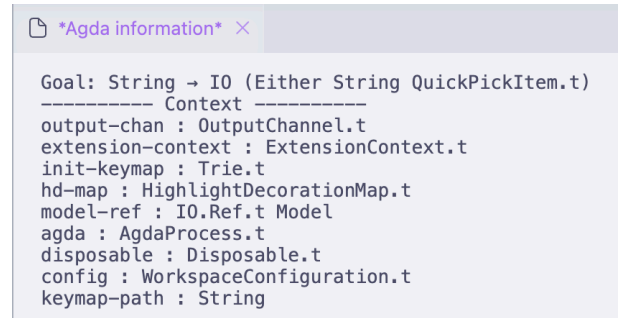
Goal display commands give information about the goal under the cursor, which is displayed in the display buffer. This information informs the user about what they need to prove and what variables are available in the current context. Examples of commands in this category are:

- the infer command, which displays the inferred type of the goal within an expression;
- and the context command, which displays the available local variables and their types, as illustrated in Figure 4.

General Commands are commands that can be executed on a module or an arbitrary expression. Examples include

- the load command, which type checks the current module and returns syntax highlighting information;
- the search about command, which searches the current module scope for functions with signatures containing some key types;
- the compile command, which compiles the current module using one of the available backends.

- The **display buffer** is a panel accompanying the editing buffers, where information about the state of the extension is displayed. For example, when the load file command is invoked the compiler shares information about its progress type-checking the current file and its dependencies. After type-checking completed, the buffer can show either information about goals – if the type-checking was successful – or it can show information about any type errors it encountered.



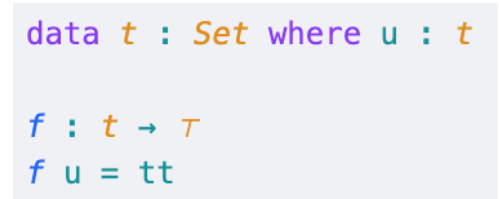
```

*Agda information* X
Goal: String → IO (Either String QuickPickItem.t)
----- Context -----
output-chan : OutputChannel.t
extension-context : ExtensionContext.t
init-keymap : Trie.t
hd-map : HighlightDecorationMap.t
model-ref : IO.Ref.t Model
agda : AgdaProcess.t
disposable : Disposable.t
config : WorkspaceConfiguration.t
keymap-path : String

```

Figure 4: The output on the display buffer after invoking the context command (C-c C-,) in Avea.

- **Syntax highlighting:** Once the current file is successfully type-checked, the compiler also provides highlighting information to the extension. Agda provides *semantic highlighting tokens*, which means that the tokens are coloured based on what they represent. For example, instead of colouring a variable black regardless of its value – Agda will colour a variable referencing a function blue. Additionally, Agda provides syntax highlighting for different error types and the locations of where they occur.



```

data t : Set where u : t

f : t → T
f u = tt

```

Figure 5: Semantic syntax highlighting in Avea. For example, the colour of the constructor u is different from the type t, even though both are variables.

- **Jump to error:** if the type-checking was not successful, then the compiler will point to the type-error it got stuck on. The extension receives information about where this error is located and moves the user’s cursor to that file at the correct position.
- **Go to definition:** The extension answers go-to-definition requests from the editor, where definition site information is provided by the HighlightingInfo message. Visual Studio Code provides a position, and the extension has to query the tokens for the currently active file to find one which has the position in its range.
- **Input mode:** All extensions have support for the unicode input mode to type unicode characters in Agda source code. This extension also has support for the input mode, in a similar fashion as the emacs-mode, where the status bar shows which characters can be typed next, the desired character can be selected using the arrow keys.



Figure 6: The status bar during input mode shows which characters have been typed, which characters can be typed next, and which variants of the current character there are.

2.2.1. Implementations

There are various implementations of Agda extensions for different editors. These implementations can use the endpoints provided by the Agda compiler for external programs to call Agda internals. The compiler has flags which open a Read-Eval-Print-Loop (REPL) environment that extensions can use to issue commands and receive responses. In general every extension has a similar high-level architecture, which is illustrated in Figure 7.

Every extension spawns one or more processes of the REPL the Agda compiler provides. When the user interfaces with the editor, the extension is notified and sends requests to Agda. Agda responds and the extension makes sure the correct updates are done to the editor.

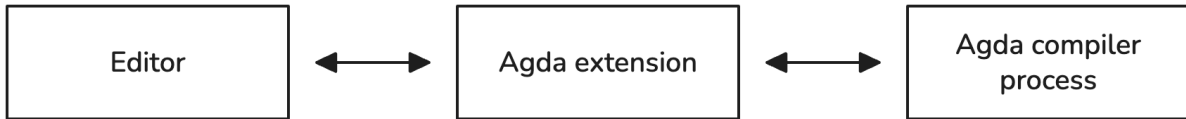


Figure 7: High level architecture of all Agda extensions. The extension is the mediator between the editor and the Agda compiler process.

Below we discuss the existing Agda extension implementations for various editors, and highlight some differences in their implementation.

- `emacs-mode` is the first Agda extension, which was developed alongside the Agda compiler. The extension is divided into two parts:
 1. The user event listeners, these are functions triggered by command hooks within Emacs. At startup, the extension opens a handle to an Agda process with the `--interaction` flag. The event listeners send different commands to the Agda compiler via this process, depending on the type of action.
 2. Agda responds with Emacs Lisp code that triggers response functions. These response functions are parameterised by data that is provided by the Agda compiler, which encodes the response data in Emacs Lisp data types.

```
(agda2-info-action
  "*All Goals*"
  "?0 : String → IO (Either String QuickPickItem.t)"
  nil)
((last . 1) . (agda2-goals-action '(0)))
```

Figure 8: Agda’s `--interaction` flag responds with Emacs Lisp code, which is executed by the `emacs-mode` extension. The `agda2-info-action` function updates the display buffer with the parameters filled in by the Agda compiler.

- `Cornelis` is an extension for Vim written in Haskell. As opposed as `emacs-mode`, it uses the `--interaction-json` flag, which has the same API as the `--interaction` flag, but responds with a JSON format instead of Emacs Lisp code.

`Cornelis` also has a different way of handling responses and managing the communication to the Agda compiler. It creates a process handle for each buffer, such that responses can be processed asynchronously, and disregard the editor’s state.

- `Agda-mode-vscode` by `banacorn` is the original extension for Agda in Visual Studio Code. It is written in ReScript, an OCaml-like language that compiles to JavaScript. It uses the `--interaction-compiler` flag, since it predates the `--interaction-json` flag.

This extension tries to emulate the behaviour of the `emacs-mode`, but VS Code has less builtin machinery for tokenisation especially when the buffer has been edited. This means the extension listens for edits to the buffer, and recalculates the positions of the known tokens to keep the highlighting working for unchecked states. Additionally, the extension also features a system to facilitate the installation of Agda.

The VS Code extension also has integration with the Agda Language Server (ALS). It is a language server for Agda created by the same author as the VS Code extension. The implementation currently only contains a hover functionality that shows the type of a term once the file has successfully been type-checked.

- Recently, a new VS Code extension for Agda has been released by Benjamin Driscoll⁴. This extension is written in Typescript – the native language of the VS Code APIs. The functionality of this

⁴<https://github.com/willtunnels/agda2-vscode>

extension is mostly the same as the banacorn’s VS Code extension, apart from some bug fixes and user interface changes.

2.3. Meta-programming

Meta-programming refers to the process of writing programs that modify or generate other programs. In this thesis we focus on static meta-programming, that is, meta-programs that are evaluated at compile-time to produce syntax trees. These syntax trees are typically spliced into the current program. Listing 1 shows a contrived example meta-program in Agda. The meta-program on the left builds up the syntax tree for an identity function, which can be spliced in using `unquote` as seen on the right.

1 <code>mk-Id : TC Term</code>	Agda	1 <code>id : {A : Set} → A → A</code>	Agda
2 <code>mk-Id = returnTC</code>		2 <code>id = unquote mk-Id</code>	
3 <code>(lam hidden</code>		3 <code>-- Expands to the following</code>	
4 <code>(abs "A"</code>		4 <code>-- at type-checking time</code>	
5 <code>(lam visible</code>		5 <code>id = λ {A} x → x</code>	
6 <code>(abs "x"</code>			
7 <code>(var 0 []))))</code>			

Listing 1: Example of a meta-program in Agda. The function on the left defines how to build up the term for the identity function, the function on the right uses the meta-program to define the identity function.

2.3.1. In Idris1

Idris1 has static meta-programming [6]. Terms can be quoted, which converts the term to a Idris1 data type representing Idris1 terms. The data type for representing terms is called `TT` (Listing 2). This data type also has a constructor for binders, which can create let bindings to introduce new variables. There is also an inverse of quotation, called unquotation. The term `%runElab m`, where `m : Elab TT`, evaluates the meta-program. The term of type `TT` that is computed is converted back into a unquoted Idris1 expression, and spliced into the current program.

Meta-programs in Idris1 are written using an embedded domain specific language (EDSL). Functions in the reflection API update the “proof” state, which consists of among others, the hole queue and the incrementally built proof term. These functions are defined within the compiler and have the ability to invoke internal function in the type-checker. For example, the `fill` function used in Listing 3 places another quoted expression in the first hole of the hole queue and unifies its type with the expected type of the hole. If this fails, the compiler presents the user with a type error.

1 <code>data Binder</code>	Idris1
2 <code>: Type -> Type where</code>	
3 <code>Let (ty, val : a) -> Binder a</code>	
4 <code>...</code>	
5 <code>data TT = Bind TTName (Binder TT) TT</code>	
6 <code> TConst Const</code>	
7 <code> ...</code>	

Listing 2: A snippet of the definition of the `Binder` and `TT` data types in Idris1. These are used to represent Idris1 terms and are the result of quoting regular Idris1 expressions.

1 <code>do</code>	Idris1
2 <code>[x, y] <- apply `(plus) [False,</code>	
3 <code>False]</code>	
3 <code>solve</code>	
4 <code>focus x; fill `(Z); solve</code>	
5 <code>focus y; fill `(S Z); solve</code>	

Listing 3: An example taken from [6]. It demonstrates how to build up the expression $Z + S Z$ using the meta-programming monad.

2.3.2. In Agda

Similar to Idris1, Agda also has static meta-programming. Agda has data types for terms, clauses and definitions, among others. The `Term` data type reflects the structure of elaborated terms in Agda. For example, `let` and `where` bindings cannot be inspected or generated using meta-programming because they are already substituted during elaboration.

Meta-programs are written in the `TC` monad, which is defined within the compiler. This monad keeps tracks of, for example, the environment, and the meta-variable state. The reflection API that Agda exposes defines a number of primitive functions that all operate in the `TC` monad.

2.3.3. In Lean4

Lean4 is another dependently-typed language and interactive theorem prover [7]. The language allows meta-programming in different ways:

- In Lean4, all top-level statements, including `def`, `inductive` are called commands. Besides these builtin commands, the user has the ability to define their own commands. The definition of a command includes a metaprogram of type `Syntax → CommandElabM Unit`. The function receives the abstract syntax tree of the argument to the command, and must produce some side effects, which include updating or querying the environment or performing IO operations. Figure 9 illustrates these components using the `check` command as an example.

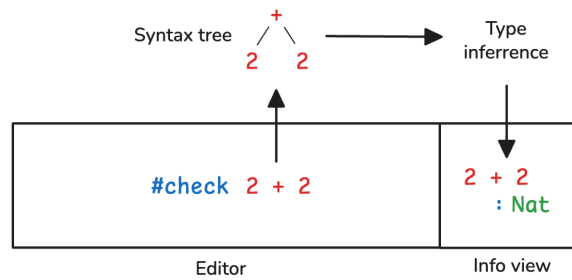


Figure 9: An example command in Lean4, and how Lean4 calls the metaprogram associated with the command.

- Macros can access the environment and the local meta-variables. Macros are very limited in terms of functionality, and can only be used to define syntactic sugars.
- Tactics can access the environment, local meta-variables and the proof state. The functionality of a tactic is defined using the `TacticM` monad, which expands directly upon the `TermElabM` monad. Custom tactics act like macros, but can be used to inspect and change the proof state in a tactic-based proof.

A meta-program in Lean4 looks similar to one in Agda, where you can call functions in a monadic non-tactic style. Whereas Agda only allows functions to be called from the reflection API, the Lean4 compiler allows meta-programs to directly call any function defined in the compiler, since it is written in Lean itself. This means that meta-programs in Lean4 can use any part of Lean4’s type checker.

Lean4 also has extensible user interface inspired by Korkut et al. [8], which allows user to execute meta-programs at edit-time and display their results in a visualisation in the infoview [9]. Widgets consist of a React component that is used to render in the info view. React⁵ is a popular web framework for Javascript, which is used to implement `lean4-vscode`’s info view. The component can call meta-programs using special hooks provided by the `lean4-vscode` info view API.

For example, Figure 10 shows an alternative implementation for the `#check` command, where the user can type an expression in the infoview (on the right), after which the type of the expression is inferred

⁵<https://react.dev/>

and displayed. When the user puts their cursor on a `#widget` command in the source file, the `lean4-vscode` extension loads the widget's React code into the infoview.

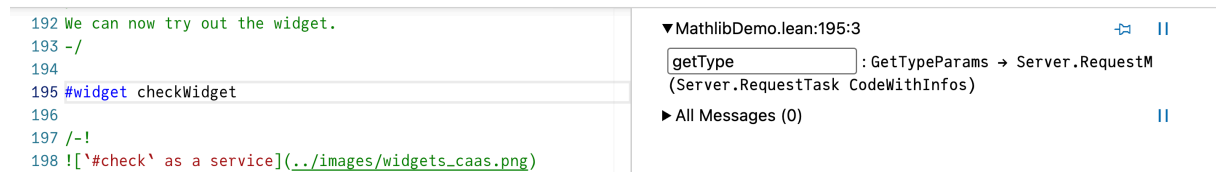


Figure 10: One of the example widgets defined on the Lean4 widgets documentation page⁶. The widget allows the user to enter an expression in the infoview and its inferred type is displayed.

⁶<https://lean-lang.org/examples/1900-1-1-widgets/>

3. Building Avea

One result of this thesis is Avea: a Visual Studio Code extension written in Agda – which compiles to Javascript – that supports the same set of features other Agda extensions also have. This chapter will discuss its implementation, and highlight some design decisions.

Avea acts as the mediator between Agda and VS Code. Agda exposes an process-based API with which Avea communicates, and Avea communicates with VS Code through the use of their Javascript API. Figure 11 shows the high-level architecture of the extension and how the different parties communicate with each other.

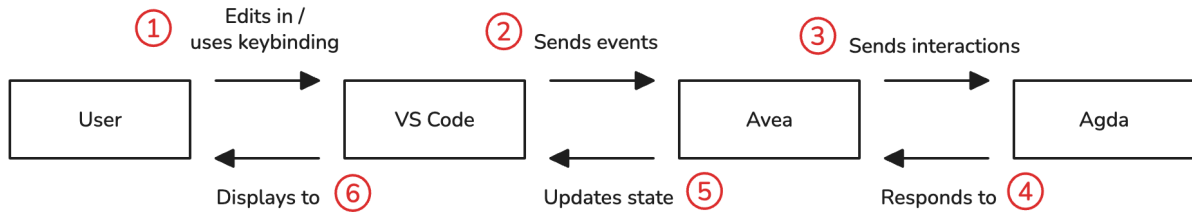


Figure 11: Communication patterns within the high-level architecture of the extension.

In order, the extension operates as follows:

1. The user makes an edit in the buffer in VS Code or triggers an action using a particular keybinding.
2. VS Code listens for keybindings configured by the extension – as discussed in Section 3.1 – and invokes the handler that is registered in the extension.
3. In the handler, the extension defines what to send to the Agda process – which typically includes which file the interactions works on, and how Agda should respond – and awaits the response. This is discussed in Section 3.2.
4. Agda responds with many different responses to the interaction that was sent. Responses can contain new highlighting tokens, information about the progress of type-checking, or formatted errors and warnings.
5. Each response is decoded into Javascript objects. Each of the parsed responses is then handled individually. This typically involves updating the internal caches, or updating VS Code’s editor buffer. Section 3.3 goes into how this is done.
6. After the caches are updated, the extension updates the editor or updates the display buffer via the VS Code API.

3.1. Running a Visual Studio Code extension

VS Code provides ways to run your extension both in debug mode as in production mode. The debug mode can be triggered from VS Code’s debug menu, which opens a new window with the extension loaded into it. To publish an extension for use in production, VS Code has a tool called `vsce`, which can package all the files needed to run the extension into a compressed binary VSIX file. This file is then published to the marketplaces.

One issue is that `vsce` only packages the files you tell it to package. Additionally, even though the `vsce` utility already compresses the extension files, VS Code recommends to keep the source code for the extension itself as small as possible. Following these guidelines, we cannot use put the `node_modules` folder – the folder containing all dependencies, both runtime and buildtime – in the extension package. For Avea, this folder can easily grow as large as a hundred megabytes, most of which is dedicated to build time tools. To remove the build time dependencies in the package, Avea uses another tool called `esbuild` – a bundler for Javascript – to combine the generated Javascript source code by Agda with

the source code of just the runtime dependencies in `node_modules`. Figure 12 shows a diagram of how the tools call each other.

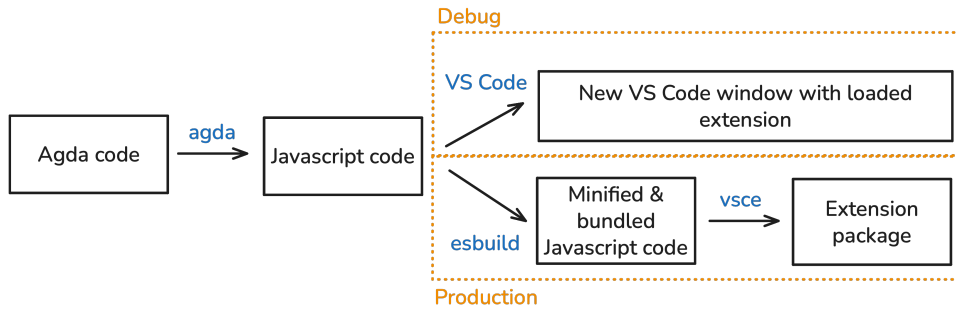


Figure 12: A high-level overview of the infrastructure required to run Avea, both in debug and production mode.

Once the extension is loaded – either in a debug window or using the production extension package – the code in the entry file is executed. The entry file sets up the dependencies – discussed in more detail in Section 4.1 – and then calls Avea’s activation function. This activation function has two purposes: it creates all of the resources that need to be created in an IO context, and it registers all of the handlers for keybindings, commands, the hover provider, the semantic token provider and the definition provider. The `activate` function serves a very similar purpose as the `main` function does in regular Agda programs, where all IO operations have to happen in `activate` or a function called from there.

3.2. Sending interactions

To send an interaction to Agda, a child process must be set up first. Avea makes sure to have one child process running at all times, so that Agda can keep its internal state about, for example, previously loaded modules. To start and manage the process, the following steps are involved:

1. Before the process is spawned, Avea needs to read the settings via the VS Code API, to read the path to the currently activated Agda binary.
2. A child process is spawned running Agda’s `--interaction-json` mode.
3. Once the process is spawned, Avea attaches different handlers. A data handler receives the output from the Agda process, which does the parsing and processing (detailed in Section 3.3). An error handler listens for unexpected closes of the process. This handler reports the error to the user, with the ability to restart the process.

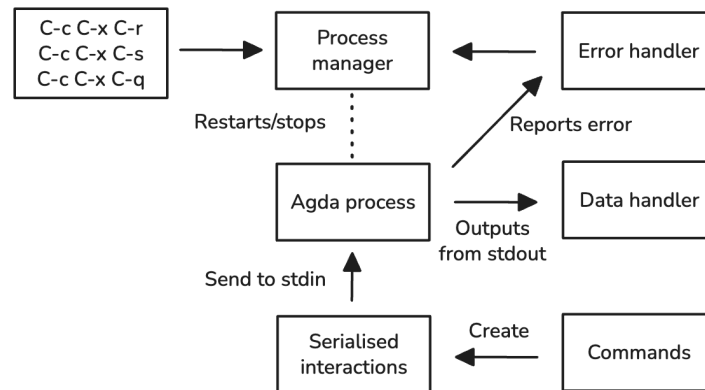


Figure 13: High-level diagram of all systems involved with the Agda process. It expects serialised interactions as input via stdin, and outputs responses to a configured data handler. The process manager makes sure the correct binary is started, and stopped or restarted when the user requests it.

When a user invokes a command that needs to communicate with the Agda compiler, the first step is to create an interaction. Interactions are encoded as stringified Haskell expressions that contain the file that is currently opened and a command. The command can also take arguments, which determines the level of normalisation, for example. Listing 4 shows how a stringified interaction looks like.

```

1 IOTCM
2   ".../Avea/Extension.agda"           -- Path to the current file
3 NonInteractive Direct                  -- How should Agda respond
4 (Cmd_load ".../Avea/Extension.agda" []) -- The command

```

Listing 4: Example of a stringified interaction sent to the Agda process when a file is loaded with C-c C-l.

3.3. Receiving and handling responses

Once we have an `AgdaInteraction.t`, Avea sends it to the Agda process. Agda responds with many JSON messages delimited by newlines per interaction. The child process data handler receives buffers of data from the standard output. These buffers may contain parts of the responses for an interaction, or multiple interactions even. Additionally, the last response might not even be complete too. The data handler therefore has the task to collect as many complete responses, and handle them. Figure 14 shows the steps in this process.

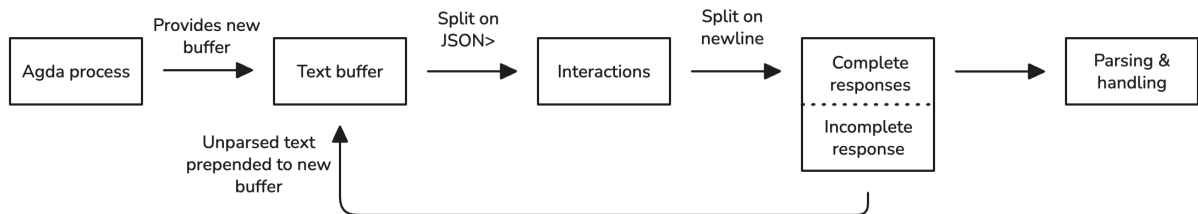


Figure 14: The “unbuffering” of Agda’s responses: Avea parses as many responses as possible and any unparsed responses are prepended to newly received buffers.

After the data handler has collected a number of responses and parsed them to JSON, it remains to parse the JSON responses to specific data types which have more specific information about the response type. These data types can be passed to a handler which calls the relevant functions from the VS Code API.

Each handler may cause edits to the buffer or perform other asynchronous actions. However, if the handlers are invoked immediately from the data handler, then there is no guarantee that the actions are executed in a deterministic order. This is because each handler acts as separate thread, which can introduce a number of concurrency problems. To mitigate these concurrency problems, Avea uses a queue to make sure only one response is handled at a time. This prevents edits from being made in the wrong order, or two edits being made at the same time, causing VS Code to throw an error.

This section will continue to describe how the handlers for three response types work to illustrate what response handlers do, and the constraints they are subject to.

Handling highlighting information

The highlighting information response contains semantic tokens. The tokens are sorted according to their positions in the source file, and then set in the internal cache. The internal cache keeps track of the latest tokens and interactions points per file. The sorting is important for the change handler to be able to efficiently search through the tokens, as will be described in Section 3.4.

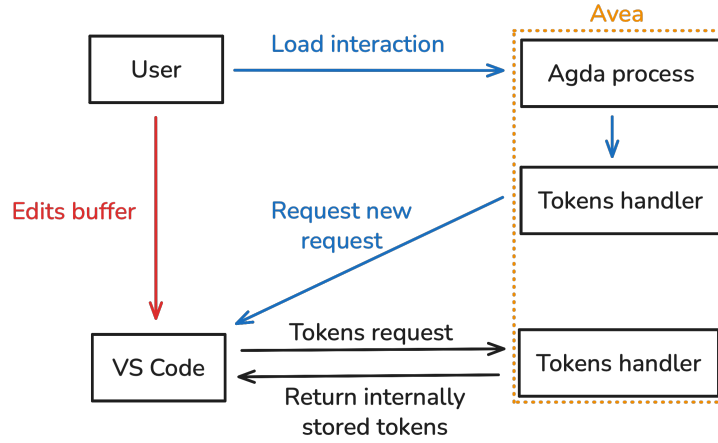


Figure 15: Diagram describing the handling of highlighting tokens. The user can request new highlighting either by making edits in the buffer (red path) or by invoking the load interaction (blue path). Avea can supply highlighting tokens on VS Code’s request via the tokens handler.

Once the tokens are updated in the internal cache, the handler will send a notification to VS Code for it to retrieve new tokens. VS Code will also automatically request new semantic tokens after a small batch of edits are made.

When VS Code requests new semantic tokens, the tokens handler registered in the `activate` function will fire. This handler simply looks for the saved tokens in the internal cache, and return semantic tokens for the tokens, and editor highlighting for errors, warnings, and interaction points. Errors and warnings are typically displayed as a change in background behind the code it applies to. For this reason, Avea has to apply editor highlighting separately for those tokens, because semantic tokens cannot be used to control the background of a piece of text.

Handling updates to interaction points

The interaction points response is included as a response to any interaction that can change the size or number of interactions points. Common interactions that returns interactions points are give (`ctrl+c ctrl+spc`), refine (`ctrl+c ctrl+r`) and load (`ctrl+c ctrl+l`). Agda sends this response to let Avea know where the latest interaction points are. Each interaction point has an id and source locations of their start and end. The id can be used to refer to a specific interaction point in subsequent interactions. The source locations are used to highlight the interaction point, and to retrieve the expression contained within. Avea keeps track of interaction points in its own local cache, and the main purpose for the interaction points handler is to update this cache with the latest information from Agda.

These interaction points can come in two kinds:

- Question marks are one character wide interaction points. The user can place a question mark in an expression to initiate a hole. It is the extension’s responsibility to expand the question mark into hole, as illustrated in Figure 16. It is therefore convenient that Agda includes question marks in the interaction points response, since these can be filtered out and expanded in the buffer.



Figure 16: Illustration of question mark expansion: when the user types a question mark in an expression (left image), Agda will signify using an interaction points response where the question marks are, so that Avea can expand it in the buffer to a hole (right image).

- Holes are interaction points that are at least four characters wide. Holes contain a potentially incomplete expression, which is delimited by goal markers `{!` and `!}`. The location of these can be put into the cache without additional processing.

Handling give actions

This response is specific to the give and refine interactions. It contains a give result – which is either a new expression, or the expression in the goal is parenthesised, or the expression is let as-is. This handler performs the following steps:

1. Depending on the give result, the handler queries the VS Code buffer to retrieve the contents of the hole.
2. The replacement expression may also contain new interaction points, which the handler needs to expand. Agda gives the new interaction points in a separate in interaction points response, which is not accessible from the handler of the give action response. However, even if it was available in the give action response, Agda returns incomplete information about the newly inserted question marks.

This requires Avea to do a manual search through the give result for expandable question marks, and replace those in the expression string to empty holes. The algorithm for this makes use of regex search to categorise parts of an expression within which a question mark should not be expanded. In particular, question marks that are part of identifiers, appear within a string, or appear within comments should not be expanded, as illustrated in Figure 17.

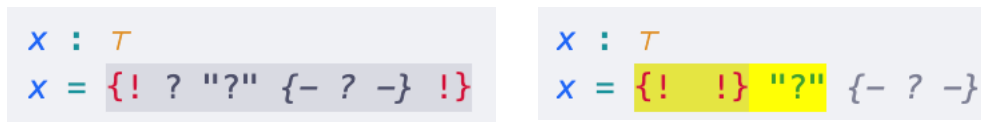


Figure 17: Expansion of question marks after a refine interaction. The left image is before the refinement, the right image shows that only the first question mark is expanded.

3. All expansions are collected into a list of edits that are made to the VS Code buffer.
4. A load interaction is issued. This means that the internal cache need not be updated since the load interaction will elicit an interaction points response that contains the most up-to-date locations of the just expanded interaction points.

3.4. Reacting to changes from the user

The semantic tokens provider is queried by VS Code, every time an edit has been made or when Avea requests VS Code to request new tokens (after a `C-c C-l`, for example). VS Code requests new tokens after edits, since there could be new tokens, or old tokens could have been invalidated. However, since Avea only receives the most up to date highlighting information after a load command, it needs to update the positions of the tokens it has in cache to provide the most accurate highlighting. Additionally, if the edits replace (parts of) tokens with new content, then tokens should be removed from the cache completely. Figure 18 shows the difference between not shifting and shifting the tokens.



Figure 18: Illustration of syntax highlighting in Avea: the left image is a loaded file. The middle image shows an edited buffer when no shifting is performed, whereas the right image shows the same edited file with shifting enabled.

Initially, Avea stored the tokens as Agda gives them. This means that no processing needs to be done when receiving the tokens, but shifting the tokens requires iterating over the entire list. Typically, a buffer is edited more times than it is loaded, so this approach can cause visual slow downs if the buffer has a large number of tokens. Another approach, which Avea uses now, is to store the tokens in sorted order. This means that the received tokens after loading need to be sorted in $\mathcal{O}(n \log(n))$ time. However, this means that after edits, Avea can find the first token that needs to be shifted using binary search in $\mathcal{O}(\log(n))$ time. After finding the first affected token, the change handler shifts the remaining tokens until the end of the line. The key observation is that only tokens on the same line after the edit need to be shifted. This prevents the visual slow downs while editing, at the cost of making the load times slightly longer.

3.5. Input mode

The input mode is one component of the extension that operates completely separately from the compiler. In fact, the input mode could even run as its own extension, since it shares no resources with the rest of Avea: it does not require access to Avea's internal caches.

Avea is bundled with a file containing a mapping from character sequences to their corresponding unicode characters. This mapping is saved in a trie structure. A trie, or prefix tree, is a search tree where each edge defines a character in the prefix of the search string. At activation, Avea converts the trie into a trie zipper, which is a trie with a focus on one node (indicated by the orange arrow in Figure 19). When the user types a character the focus changes to a node downwards, or when the user hits backspace, the focus moves upwards. When a character sequence has multiple associated unicode characters, the arrow keys move the focus in a list zipper to left or right.

While the user is typing characters, the trie is traversed until no character matches anymore. Then the sequence of characters is replaced in the buffer by the unicode character under focus, and the input mode stops listening for new characters. Figure 19 illustrates what happens to the trie zipper as the user interacts with the input mode.

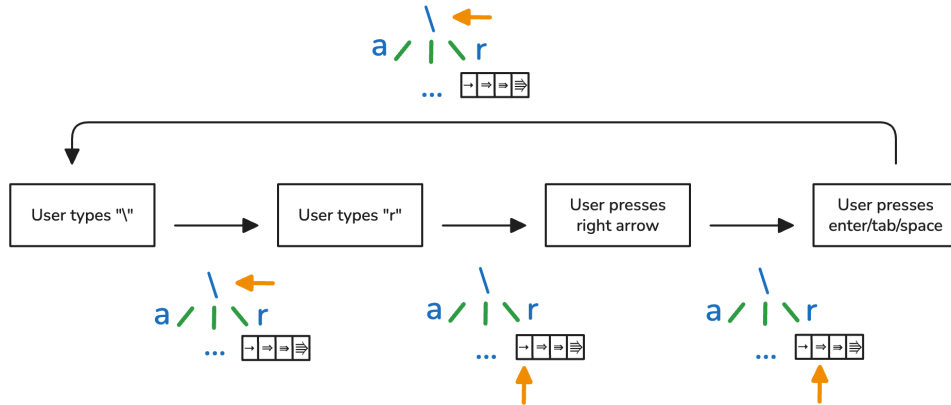


Figure 19: The changes to the input mode's trie zipper as the user types characters.

The reason for using a trie zipper is that zipper structures are easy to implement for traversing a tree-like structure up and down in a functional language. In imperative languages, a tree can be defined using pointers, but since these are not available in Agda, a tree zipper must instead be used.

4. Challenges with programming in Agda

To be able to answer the first research question “Is Agda a suitable language to write practical programs with?”, we have to reflect on some of the major challenges we faced during Avea’s development. In this chapter, we will discuss those challenges, and the solutions we have applied in Avea. We will also generalise these solutions to other types of programs, and use that as an argument to answer the first research question.

4.1. Importing Javascript libraries

Avea uses some external Javascript libraries that need to be imported, in order to function. Usually in Javascript, the `import` statement is used to import libraries. However, the current release of the Agda compiler does not provide a way to import Javascript libraries. Instead, Avea relies on a separate Javascript file to set the dependencies in a global variable available across all modules, and then call the `activate` function, as shown in Listing 5.

```
1 import Main from "./jAgda.Avea.Extension.mjs";
2 import * as vscode from "vscode";
3
4 export function activate() {
5   Object.defineProperty(globalThis, "AveaImports", {
6     value: { vscode, /* and some other libraries */ }
7   });
8   Main(); // Call the main function from the extension manually
9 }
```

Listing 5: A Javascript file importing libraries and making them globally available before running the Agda code.

While this approach works in the current version of Agda, it must be noted that it is not fully safe to use everywhere. To illustrate, the code in Listing 6 below throws an error at runtime. This is because the evaluation of `imported-string` for the value of `s` happens before the `activate` function is called. This means that `AveaImports` is not defined when `imported-string` is evaluated, and therefore `someString` cannot be accessed, throwing a runtime error.

```
1 postulate imported-string : String
2 {-# COMPILER JS imported-string = AveaImports.someString #-}
3 s = imported-string <> "!"
```

Listing 6: Illustration of using `AveaImports` at the top-level, which causes runtime errors.

This issue can be solved by wrapping `imported-string` into `I0` as illustrated in Listing 7. This delays the evaluation of `AveaImports` until it can be used, which, in the extension, is guaranteed to be after `AveaImports` is defined. Even though this approach works, it only allows top-level definitions to be used in effectful functions, even if the values themselves might be pure. This leads to unnecessary colouring of functions to be `I0`, or passing the top-level definitions as parameters to functions that need them, undermining the use of the top-level definition.

```

1 postulate imported-string : IO String
2 {-# COMPILER JS imported-string = async () => AveaImports.someString #-}
3 s = fmap (<> "!") imported-string

```

Listing 7: Using AveaImports at the top-level without causing runtime errors by delaying the access to someString through the use of IO.

A similar issue has already been solved for the Haskell backend through the `FOREIGN GHC` pragma. The `FOREIGN GHC` pragma allows the user to insert arbitrary code at the top of the output files. However, the Javascript version of this pragma, `FOREIGN JS`, is still unimplemented at the time of writing. Implementing the `FOREIGN JS` pragma requires very few changes to the Agda compiler, and we created a fork of the Agda compiler that includes support for the pragma⁷. An example use of the pragma is shown in Listing 8.

```

1 {-# FOREIGN JS import { someString } from "library" #-}
2 postulate imported-string : String
3 {-# COMPILER JS imported-string = someString #-}
4 s = imported-string <> "!"

```

Listing 8: Using the `FOREIGN JS` and `COMPILER JS` pragmas to import a variable from a Javascript library.

The `FOREIGN JS` pragma allows the modules to import their Javascript dependencies without requiring a separate file to bind them to the global scope, and allows these dependencies to be used at the top-level without requiring IO. This is because it allows the import of `someString` in the same file where the `imported-string` variable is defined. Given that the `FOREIGN JS` is placed before `imported-string` is used, `someString` can be correctly resolved, and the code runs without any errors. Unfortunately, since the implementation of the `FOREIGN JS` pragma has not been finalised and released yet, Avea still makes use of the imports object approach.

4.2. Handling asynchronicity

By default Agda code compiled to Javascript expects IO operations to use continuation-passing style, though the builtin module does not provide the monadic operations: `pure` and `_>=_`. When compiling, the Agda compiler looks for a main function, and if it exists, it inserts `main() => {}` at the end of the file. The standard library author, therefore, has a choice how to implement these functions as long as calling the function starts the IO operation.

Initially, the extension made use of continuation-passing style for IO (provided by the `Iepje` library⁸). However, the VS Code API exposes some functions that return promises: a type that modern Javascript uses to handle asynchronous computations. A promise represents a computation that has already started and will return a value in the future, an example of which can be found in Listing 9.

```

1 const delay = (time, f) => new Promise((resolve, reject) =>
2   setTimeout(
3     () => resolve(f()),
4     time
5   ));
6 delay(1000, () => 10).then(n => console.log(n)); // Prints 10 after 1 second

```

Listing 9: Example function that uses promises that delay a computation and waits for the computation to finish to print the result.

⁷<https://github.com/agda/agda/issues/8429>

⁸<https://github.com/lawcho/iepe>

To be able to use these functions from the VS Code API, Avea requires a different definition for `I0`, where instead of compiling to continuation passing style, we use lazy promises. Using just promises for the `I0` monad – with the new constructor as pure, and `.then` for `_>=_` – means that any term of type `I0 A` has already started executing. Since Avea’s implementation makes abundant use of “lazy” `I0`, where storing the `I0` action does not mean it has already started executing, we have decided to define `I0` as functions of Typescript type `<T>() => Promise<T>`.

However, in order to avoid stack overflows when, for example, traversing large lists with the `I0` monad, we need to make use of the syntax sugar that Javascript has for promises: `async/await`. As seen in Listing 10, an implementation with `async/await` is a little more verbose, but reduces the stack depth of nested `I0` operations compared to using `.then` directly – which adds a stack frame every time it is called.

<pre> 1 ma => f => async () => { 2 const a = await ma(); 3 return await f(a)(); 4 }</pre>	<pre> 1 ma => f => () => 2 ma().then(a => f(a)())</pre>
--	---

Listing 10: Juxtaposition of an implementation of `_>=_` with stack depth linear to the nesting depth of the `_>=_` operations (left), and an implementation with stack depth linear to the number of evaluated `_>=_` calls (right).

While Avea’s definition of `I0` works, it could still be generalised. Instead of defining all `I0` to be asynchronous, it is possible to subdivide `I0` into `SyncI0` and `AsyncI0`. `SyncI0` would be `I0` using the continuation passing style, only allowing synchronous operations. `AsyncI0` would use promises. This means that any `SyncI0` operation could be lifted to a `AsyncI0` operation quite trivially. Conversely, the concurrent queue used to force sequential evaluation of `AsyncI0` operations could be seen as a way to run `AsyncI0` actions in a `SyncI0` environment.

4.3. Mitigating stack overflows

The Agda compiler does not support tail-call optimisation by itself. Compilation of Agda programs to Haskell is usually faithful enough that tail-call recursive programs in Agda are also tail-call recursive in Haskell. GHC can then do its tail-call optimisation, and by extension, the Agda can use tail-call recursion without stack overflows.

However, most major Javascript engines do not support tail-call optimisation. Since recursion is the only method of traversing a data structure in pure Agda, this means that the stack size determines how deep we can traverse a data structure in one pass. For Avea’s implementation this poses a serious problem, since the responses from Agda containing the highlighting tokens grow linearly with the size of the source file that is being checked. For relatively small source files, we already run into the stack limit.

To mitigate this, we need to use the JavaScript FFI to define “stack-safe” functions that can traverse large data structures without increasing the stack depth per element. Agda allows us to provide a definition that is used by the type-checker, and another separate one that is used in the compiled output. This means we retain the ability to reason over the functions, as well as removing the worries about the stack depth.

```

1 reduce-right : B → (B → A → B) → List A → B
2 reduce-right b f [] = b
3 reduce-right b f (x :: xs) = f (reduce-right b f xs) x
4
5 {-# COMPILER JS reduce-right = a => A => b => B => b => f => xs =>
6   xs.reduceRight((ac, e) => f(ac)(e), b) #-}
```

Listing 11: The implementation of the reduce-right function both for Agda’s elaborator and for the compiled output. The Javascript definition offers stack-safety.

While this method does resolve stack-safety issues, we do have another problem. Agda takes the COMPILER JS pragma and inserts it unchecked in the compiled output. Therefore, there is no guarantee that the definition in the pragma adheres to the type declaration, and that the pragma definition has the same semantics as the Agda definition.

A pragmatic solution to this “consistency checking” problem is to create a separate program that does the following:

1. For each test case, the program requests a fully qualified function name, and test inputs for each relevant parameter. The program will compile the library and find the correct Javascript definition in the compiled output.

```

1 run_consistency_tests
2   name: "Data.List.map"
3   inputs: cartesian_product test_functions, test_lists
4   erased_params: 4
5   meta_information: "{A = Nat} {B = Nat}"
```

2. The given inputs are applied to the function, taking the erased parameters into account, and then they are serialised together with the output for the next step.
3. The outputs from the compiled JavaScript code are inserted into a temporary Agda module that prompts the elaborator whether the the same function call gives the same output. In particular, the tester creates Agda source files with the structure described in Listing 13.

```

1 module Test where
2   -- Import the module containing the function we want to check.
3   open import Data.List
4   open import Agda.Builtin.Equality
5   -- For each test case, there is an assertion that the Agda definition
6   -- normalises to the same output as the Javascript definition did.
7   _ : map (λ x → x + 1) (1 :: 2 :: 3 :: []) ≡ (2 :: 3 :: 4 :: [])
8   _ = refl
```

Listing 13: Agda file for checking consistency once the output of the JS definitions is collected.

If this file type checks, it means that for each test case, the Agda definition is return the same results as the Javascript definition. Otherwise there might be issues in i.a. the number of parameters, different semantics, or type errors in the Javascript definition.

With well-chosen test cases, there is a reasonable degree of consistency between the Agda and Javascript definitions. There are some limitations to this tool, however, especially when testing functions operating on more complex data types, or functions using type classes. Data types and type classes are compiled to more complex objects, and may make use of external functions whose structure and generated names are quite hard to predict.

The solution to this would be to write the consistency tests in Agda. We can take inspiration from Haskell’s tasty testing framework⁹. We define an Agda program that contains (grouped) consistency tests. For example, we want to check consistency for `map`. We define a test case consisting of an input function `f` and an input list `xs`. Then we evaluate `map f xs` using the reflection API, the result of which is `ys`. Then we compile the expression `map f xs` to Javascript. Running the compiled Javascript code, will evaluate `map f xs` using the `COMPILE JS` version of `map`. The result of this is checked for equality to `ys` (using some user-defined equality function) and report back to the user. Figure 20 shows the commutative diagram that is checked.

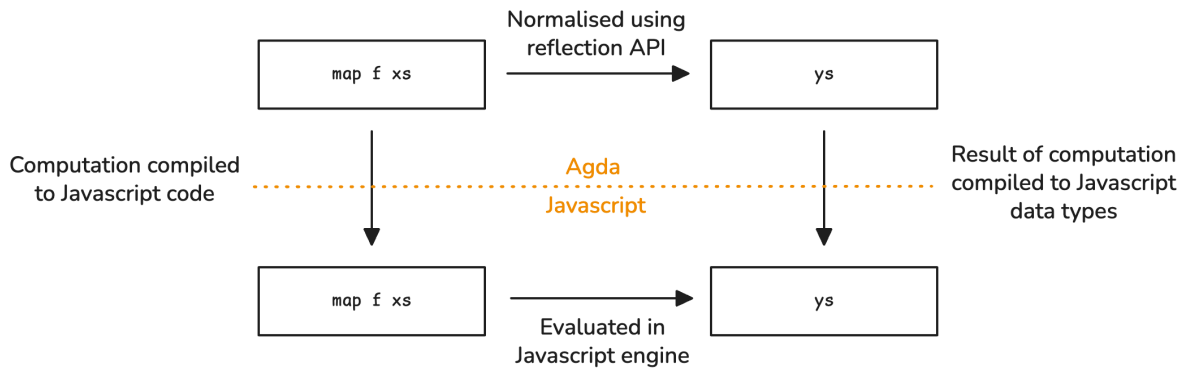


Figure 20: Commutative diagram for consistency checking the expression `map f xs`. The goal is to verify that the Agda definition and the `COMPILE JS` definition of `map` produce the same result `ys` for the given inputs `f` and `xs`.

4.4. Evaluation

With the major implementation challenges discussed, we can answer the first research question: “Is Agda a suitable language to write practical programs with?” Yes, we believe that it is. Our arguments are split over the target language a developer uses:

- Programs targetting Haskell can benefit from the relatively predictable compilation, which handles tail-call optimisation to do recursion over large data structures and imports. Additionally, the Foreign Function Interface with Haskell is easy to use through the use of the `FOREIGN GHC`, and `COMPILE GHC` pragmas.
- Targetting Javascript requires some additional infrastructure and programming disciplines. Imports need to be handled outside of Agda currently. Though the `FOREIGN JS` pragma will solve the remaining issues that this approach to imports has. In general, we think more development on the Javascript backend is beneficial, for example, a simpler or more efficient encoding of data types and records. Javascript is a versatile language that can run in the browser, as well as in desktop applications. Compiling Agda to Javascript allows developers to combine formal reasoning with new practical use cases that Haskell cannot fulfil, for example, formally verifying parts of a web application.

Furthermore, Javascript engines do not typically have tail-call optimisation, which means recursion should be used sparingly. Instead, developers should use functions such as `map` or `reduce` that are stack-safe.

For more complex data types, developers should implement their own stack-safe traversal functions. The `COMPILE JS` pragma allows developers to define a stack-safe version of a function, and a regular Agda definition defines a version that can be reasoned about at the type-level. Using a consistency checker, we can give stronger guarantees about whether the two definitions behave the same in the given test cases.

⁹<https://hackage.haskell.org/package/tasty>

5. Adding extensibility

In this chapter, we will look into making Avea extensible. For this we have decided to heavily depend by previous work on editor reflection by Korkut and Christiansen [8]. They introduced the concept of editor reflection in Idris1 – a language with a similar meta-programming library, and a similar compiler structure as Agda.

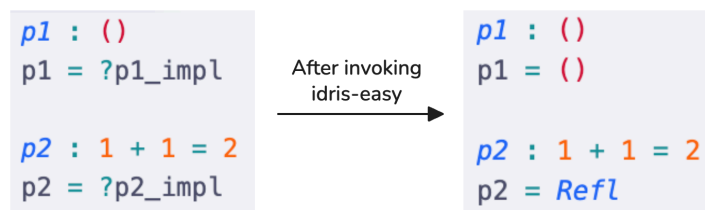


Figure 21: Example use of the `idris-easy` editor action defined below. The action can be executed on a hole, and it is filled with the returned expression from the meta-program.

In brief, editor reflection allows users of Idris1 to write meta-programs that can be executed from the editor. Listing 14 lists the code used to implement the example editor action taken from [8], whose functionality is shown in Figure 21. The meta-program in Listing 14a has a goal name parameter and attempts to solve the goal by inspecting its type. If the type of the goal is unit, it will return unit (line 6), if the goal type is an equality type and the types and values are convertible, then the reflexivity constructor is returned (line 10), otherwise the goal is not trivially solvable and an error is returned. This rudimentary solver is invoked by the editor using the Emacs Lisp support code (also called extension code) in Listing 14b. This code defines the editor action and what arguments (line 6) to send to the meta-program.

```

1  %editor
2  easy : TTName -> Elab TT
3  easy n = do
4    ty <- getType n
5    case ty of
6      `(() : Type) => pure () : ()
7      `(=) {A~a} {B~b} ~x ~y => do
8        converts a b
9        converts x y
10     pure `(Refl {A~a} {X~x})
11    _ => fail
12    [TextPart "Cannot solve"]

```

Listing 14a: Idris1 meta-program that tries to solve a trivial goal.

```

1  (defun idris-easy ()
2    "Example editor action"
3    (interactive)
4    (idris-elab-hole-arg
5     "easy" (list
6             (idris-name-at-point))))

```

Listing 14b: Emacs Lisp support code to call the meta-program on the left from within the Idris1 editor.

While the design by Korkut and Christiansen makes for an extensible editor tooling, users of Idris1 still have to write Emacs Lisp code to create an interface between the editor action and the editor itself. This has two drawbacks:

1. There is a potential source of bugs where the editor action returns an unexpected data type from the Emacs Lisp support code's perspective, or the support code sends an unexpected data type in the editor action's perspective. There is no way currently to constrain both ends to use the same data types.

2. Users need to learn the Emacs Lisp DSL. This might not be a big problem for smaller editor actions, however, more advanced editor actions might require more processing on the editor side, which requires more advanced knowledge of Emacs Lisp.

These drawbacks can be solved by allowing the support code to be written in the same language as the editor actions. Since Avea is written in Agda already, and Agda has similar meta-programming capabilities as Idris1, Agda is perfect candidate for this. The example from Listing 14 would be translated to the Agda program in Listing 15.

```

1  easy : EditorAction.t
2    (Term → TC Term)
3  easy hole = editor-action $
4    inferType hole >=> λ where
5    (def (quote τ) [] →
6      pure (quoteTerm tt)
7    (def
8      (quote _≡_)
9      ( (arg _ _) :: (arg _ A)
10       :: (arg _ x) :: (arg _ y)
11       :: [])) → do
12      A' ← unquoteTC {A = Set} A
13      unify x y
14      x' ← unquoteTC {A = A'} x
15      pure (quoteTerm $
16        refl {A = A'} {x = x'})
17      _ → typeError $
18      strErr "Cannot solve" :: [])

```

Listing 15a: Translation of the easy action described in Listing 14a into Agda. Since Agda does not have attributes (such as %editor), the type signature is annotated.

```

1  agda-easy : AveaExtension.t τ
2  agda-easy = do
3    h ← hole-under-cursor
4    term ← get-hole-content h
5    new-term ← call-editor-action
6      easy [ term ]
7    fill-hole-with new-term h
8
9  registrations : List Registration.t
10 registrations
11   = hole-cmd agda-easy
12   :: []

```

Listing 15b: Translation of the “glue” code connecting the editor action on the left with Avea. The special registrations variable will be loaded by Avea to discover all extensions.

To implement an editor with editor reflection fully in Agda, we can extend Avea and the Agda compiler to support editor reflection, and for this we identify the following stages of development. These stages are ordered such that each stage adds new functionality on top of the previous stages. Even though the implementation of these stages goes beyond the scope of this thesis, we will still describe a rough design for each stage.

- In Section 5.1, we will introduce a basic embedded domain specific language (EDSL) in Avea for listening for events in VS Code and handling them, and adding new edit-time functionality extensibly without making changes to the Agda language or the Agda compiler. We will call these **extensions** in Avea. The new functionality of this system will be rather limited: this API would allow people to write snippets or automate a sequence of hole commands for example in an extensible manner.
- In Section 5.2, we will discuss being able to call meta-programs, which we call **editor actions**, in the Agda compiler. The implementation of this is mostly based on the work of Korkut and Christiansen, but applied on the Agda compiler. Adding editor reflection to the current version of the Agda compiler allows for non-syntactical expression manipulation in holes. Examples include, simplification of programs written in an EDSL, customised code snippets (e.g. generating boilerplate for type class instances), and proof search actions.

- In Section 5.3, we will discuss why the current reflection API does not include many high-level language constructs (such as `let`, `where`, `do`) or provide source locations. Incorporating these into the reflection API would allow for syntactic changes and more precise edits in the buffer. This enables use cases such as lintings and warnings, which can be executed on every load of a file, and display squiggles in the buffer at exact locations.
- Finally, in Section 5.4, we will briefly discuss the possibility of porting Lean4’s widgets to Avea and Agda. This will allow more advanced user interfaces in the display buffer, while still not needing to leave the confines of Agda.

5.1. Start of the Avea extension EDSL

Users can always extend Avea by forking its source code, adding a new feature and recompiling the code into a VS Code extension. However, this is a tedious process, which is not very extensible. Instead, Avea needs a way to load extensions, without editing or recompiling Avea.

To do this, Avea can load a specific Agda project – with a `.agda-lib` file to declare dependencies – somewhere on the user’s file system. This library requires one specific exported module, which Avea loads and executes. This special module contains all hooks and extension actions.

```
1 module Avea.Extensions.Hooks where
2
3 registrations : List ExtensionRegistration.t
4 registrations = on-hook load ... :: on-cmd ... :: ... :: []
```

Listing 3: Startup file for Avea extensions. The registrations variable is a list that says when a function should be called after what event.

The hooks are registered by calling constructors of the `ExtensionRegistration.t` type. The hooks are defined in Avea, and listen for events from a number of relevant VS Code features. For example, extension code can register a command in the command palette in VS Code or listen to lifecycle hooks that are executed after a particular event, e.g. on a successful load of a file.

```
1 module ExtensionRegistration where
2   data t : Set where
3     on-hook : AveaHook.t → ExtensionAction.t T → t
4     on-cmd : AveaCommand.t → ExtensionAction.t T → t
5     on-menu-button : MenuButton.t → ExtensionAction.t T → t
```

Listing 4: Definition of `ExtensionRegistration.t`, which is used to define hooks.

The extension actions that are executed when a hook triggers, are programs written in the `ExtensionAction.t` monad. The `ExtensionAction.t` monad is a `State` monad where the state is Avea’s model. However, the monad does not allow any field in the model to be modified: changes or accesses must be done through the defined functions in the Avea library. This way extensions cannot interfere with Avea’s internal system, and potentially compromise its other functionality. So examples of functions that operate in this monad are described in Listing 5.

```

1 hole-under-cursor : ExtensionAction.t Hole.t
2 get-hole-content : Hole.t → ExtensionAction.t HoleTerm.t
3 get-hole-range : Hole.t → ExtensionAction.t Range.t
4 replace-hole-with : HoleTerm.t → Hole.t → ExtensionAction.t T
5 write-to-current-buffer : String → Range.t → ExtensionAction.t T
6 send-agda-interaction : AgdaInteraction.t → ExtensionAction.t AgdaResponse.t

```



Listing 5: Signatures of functions in the proposed Avea extension EDSL.

The implementation of extensions in Avea should not pose too many problems. It requires:

- Implementations of the functions described in Listing 5. These functions already exist within the current implementation, but need to be adjusted to work within the `ExtensionAction.t` monad.
- Each hook should be triggered at the appropriate time. For registrations involving commands or buttons, they should first update the UI and let the callback call the `ExtensionAction.t`. Each life cycle hook just needs access to the registration list to call the functions subscribed to the hook in question.
- Adding a new “reload extensions” command in Avea, which recompiles the Agda project containing the registrations and loads the compiled Javascript code and updates the registrations in Avea’s model. This is needed when an extension is added, removed or, while developing an extension, its code has changed.

Use cases

The extension EDSL does not unlock very powerful edit actions, apart from some automations of existing sequences of Agda interactions. However, there are still some edit actions that were not possible to implement before, which are listed below.

- *Caching for proofs by reflection*: proof by reflection is a technique where functions mechanically generate proofs for a theorem [10]. Computing these proofs for large proof trees can be expensive, especially considering that these proofs may not change often. Therefore, it might be more efficient to inline the proof tree in the source to avoid doing the proof by reflection at elaboration-time. Inlining the proof tree could be implemented as an editor action in our extension.
- *Record and replay*: Listening to hooks of all interactions, it is possible to record how a definition is “created” by recording the use of Agda interactions. These can be saved and serialised, and later on replayed on another definition to recreate it, or exported for didactical purposes.

5.2. Edit-time actions in Agda

Instead of modifying the compiler to add editor features, Korkut and Christiansen [8] have implemented an extensible editor for Idris1 where editor commands can be defined within Idris1 itself using the metaprogramming library. The procedure they used to implement the editor extensions can be translated quite well to Agda. As illustrated in Figure 22, the procedure consists of designing a user-friendly API to call editor actions in a type-safe manner (Part 1), defining the communication protocol between the interaction mode and Avea (Part 2), and running editor actions in the compiler (Part 3).

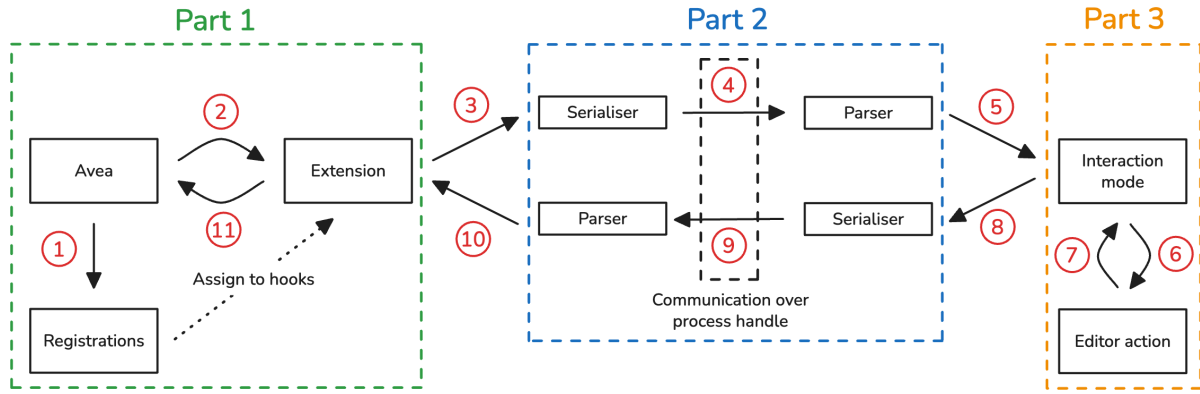


Figure 22: Complete diagram showing the steps that are taken to call an editor action in the Agda compiler from an Avea extension.

1. Designing the API to call editor actions

We can extend the extension EDSL proposed in Section 5.1 to include a way to call editor actions. Calling an editor action is done in a similar way to calling other commands in Agda’s interaction mode. This means that it is possible to reuse a lot of the logic for sending interactions and receiving and handling responses in Avea, and the logic from Section 5.1 for making the responses available in extensions.

To call an editor action on a hook, one needs to register an extension on the hook (step 1 in Figure 23). Avea calls the extension (step 2) when the hook triggers, and the extension can call the editor action using by communicating with the interaction mode (step 3). The result is communicated back, and the extension can call Avea internals to display the returned data (step 10 and 11).

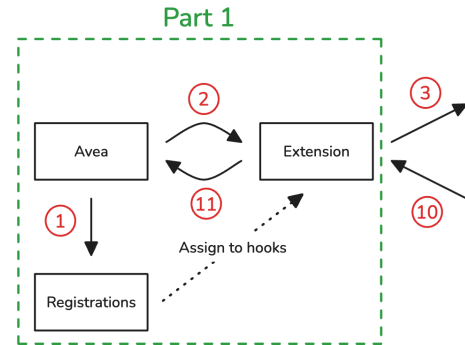


Figure 23: Diagram of Avea running extensions calling editor actions.

Editor actions are co-located with the Avea extensions. This means that we can write meta-programs that can inspect the type of the meta-program, and find type class instances at compile time. Therefore metaprogramming allows us to provide a type-safe API to call editor actions, even though they are executed in a different environment from the Avea extensions.

2. Defining the communication protocol

Editor actions are executed in the Agda compiler. However, they are triggered by the extension and receive arguments from there, and the result of the editor action also needs to be communicated back to the extension. As mentioned in Section 3, Avea communicates to the Agda compiler through its interaction mode. Avea sends serialised Haskell expressions to the interaction mode (step 4 in Figure 24), and it responds with JSON objects (step 9).

This means that to send values to editor actions, it is easiest to serialise them to a string, and embed

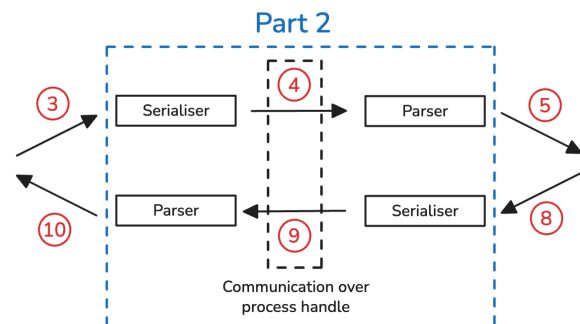


Figure 24: High-level diagram of the communication between Avea and the compiler, where Avea is on the left and the Agda compiler on the right.

those strings in the Haskell expressions. And for receiving values, the reverse needs to happen: strings in the JSON responses need to be deserialised to a value in Avea. The serialisation and deserialisation of values of type `A` can be defined a type class: `Encodable A M`.

However, the representation in Avea of a type might not correspond well to what editor actions use. An example is expressions: Avea only has access to stringified high-level Agda source code, whereas meta-programs usually expect parsed and elaborated terms. The `Editorable` type class captures this convertibility between different types. It combines two `Encodable` instances: one for the representation in Avea (step 3 and 10 in Figure 24), and one for the representation in the Agda compiler (step 5 and 8).

```

1 record Encodable (A : Set) (M : Set → Set) : Set₁ where field
2   to   : A → String
3   from : String → M A
4
5 record Editorable (A B : Set) : Set₁ where field
6   runtime : Encodable A Maybe
7   action  : Encodable B TC

```

Listing 6: Possible definition for the `Encodable` and `Editorable` type classes used for communication between Avea and the interaction mode.

3. Running editor actions in the compiler

Two major changes are needed to the Agda compiler to add support for running editor actions.

- A new endpoint in the interaction mode needs to be implemented. As any endpoint, it always receives information about the file to operate on. Additionally, editor actions require the name of the action, and a list of serialised arguments.

The handler of the endpoint, loads the requested module, searches for the editor action by name in the top-level scope, and then runs the metaprogram in the editor action (step 6 in Figure 25). The editor action is given the deserialised arguments, and produces a result (step 7). This result is serialised again and send back as an Agda response to Avea.

- The compiler needs to provide some primitives to implement some instances of `Encodable`. In particular, Avea can send high-level Agda code, that needs to be parsed and elaborated before it can be used in an editor action (step 5). Returning an Agda expression has to do the reverse: it should be delaborated and serialised before sending it to Avea (step 8). Parsing, elaborating, delaborating and serialising of Agda code within Agda is currently not possible, hence the need for the new primitives.

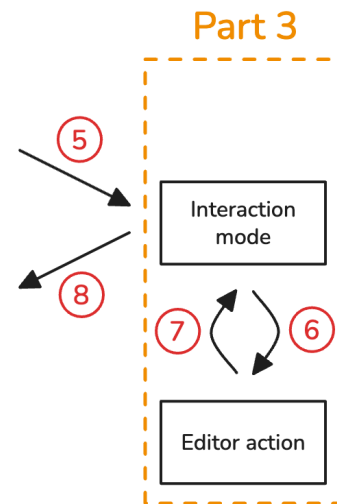


Figure 25: High-level diagram of the system to execute editor actions in the Agda compiler.

Use cases

The editor reflection system as we have described it above allows for more powerful edit-time features. In particular, users can now combine the functionality of Avea’s extension EDSL and the functionality of Agda’s reflection API to write new tooling. This means that editor reflection allows users to transform expressions in holes or fill holes using a meta-program. Some examples include:

- *Hoogle-like lemma search*: this feature requests all names available in the current scope of the cursor and tries to unify them with a given type (or the requested type of the hole). This feature needs to communicate a list of types back to the extension, which is then displayed in the display buffer.
- *Re-implementation of some existing interactions*: With the ability to inspect the local context, it is possible to replace interactions such as `give (C-c C-SPC)`, `refine (C-c C-r)`, `goal environment (C-c C-.)`, `infer (C-c C-d)` and `normalise (C-c C-n)`. These interactions can be implemented using the reflection API, and therefore their implementation can be moved from being in the compiler to being editor actions in a standard library.
- *Custom automatic proof search*: Kokke and Swierstra [11] have implemented an extensible proof search mechanism using just Agda’s reflection library. Editor reflection allows users to call this proof search library at edit-time, which has a number of advantages, as remarked by the authors in the discussion. Firstly, the proof search does not need to run every time the module is compiled, since its resulting program can be spliced into the source code of the program. Secondly, the resulting program can also contain new holes, which opens the possibility for the proof search mechanism to return incomplete terms.

5.3. Changing Agda’s meta-language

Editor reflection unlocks allows users to reason about and change the semantics of their code at edit-time. However, the current reflection API in Agda does not allow for inspecting syntactic elements of their code. Recall from Section 2.3.2 that `let` and `where` clauses are all substituted in reflected terms. The benefit of this is that it is closer to the internal representation of terms in the compiler, and that the data type describing needs less constructors. Additionally, because the compiler does a lot of substitutions, it is difficult to keep track of the source locations of reflected terms. The compiler changes the structure of the code from its concrete syntax, and therefore there exist many nodes in the abstract syntax tree that do not exist in the concrete syntax.

While for regular meta-programming, the semantics of the code is the only concern for programmers, because the generated code from the meta-programs is not typically displayed. This means that making changes to syntactical constructs of a program is futile, since in the eyes of the developer, nothing changes about the result of the final program. However, since editor reflection allows users to make change to the source code of their programs, being able to reason about and make changes to the concrete syntax becomes relevant.

Korkut and Christiansen also mention this in their future work in [8]: “In the future, it would be interesting to explore representations of the syntax of high-level Idris that are robust in the face of change and extension.” With a representation of high-level Agda – that includes source locations – in the reflection API, it is possible to unlock a lot more potential of editor reflection. In particular, it would be possible to be more precise about the location of where edits should be applied or where warning squiggles should appear. This is the driving mechanism behind the following examples:

- *Unused variable linter*: This is a feature that many more mainstream languages have, where unused variables are greyed out or marked as a warning for being redundant. The editor action part of the linter can traverse the structure of the program, and keep track of usage per binder, when finally, it can return the locations of the unused binders. The extension code can trigger on every load, where

it will call the editor action, and put squiggles under every returned source location with a hover information message.

- *Style checkers*: Taking the idea of the previous linter a step further, style checkers analyse the structure of the code for general *anti-patterns*, that is, obvious structures that can be improved. These style checkers can also make use of VS Code’s suggested fix button in the hover view. This button can trigger another editor action, which replaces the code violating style rule with another piece of code with the preferred style.
- *Re-implementation of the case split interaction*: Given the possibility to reason about source locations, it is now possible to also re-implement the make case interaction (C-c C-c). In addition, it would also be possible to improve its functionality, by allowing it to create cases for lambda parameters for example.
- *Refactorings for dependently-typed programs*: We can also implement refactorings inspired from Barwell et al. [12]. They introduce refactorings such as adding an index to a data type definition, or relating two constructor parameters using a dependent type. This feature could be used to its full potential with bindings to VS Code’s refactorings menu. This menu allows the user to see a preview of the changes that a refactoring applies and to select multiple refactorings to see how they interact, and give the desired result.

5.4. Port of Lean4’s user widgets to Avea and Agda

As mentioned in Section 2.3.3, the `lean4-vscode` extension allows custom widgets to be embedded in the infoview. Widgets are defined using React – a web framework in Javascript – and allow any user interface to be displayed in the info view. Widgets in Lean4 can call meta-programs, which is how Lean4 has designed editor reflection. It is also possible to allow arbitrary user interfaces relying on meta-programs in Avea’s display buffer.

The architecture of Lean4’s widgets can be described by the diagram in Figure 26 [9]. There are two points of special note:

1. Widgets can directly communicate via LSP to the Lean4 compiler. If the widgets want to evaluate metaprograms, then they can use the injected Lean4 RPC library to call metaprograms. The RPC library only allows function calls to meta-programs marked with the `@[server_rpc_method]` attribute.
2. The React code for the widgets is stored as a string in the Lean source file. When a widget should be loaded, this code is sent to the infoview, which embeds the React component into its own React app. The infoview manages all communication to and from the extension: if a message is native to the infoview, it handles it itself, otherwise it can dispatch the message to the widget.

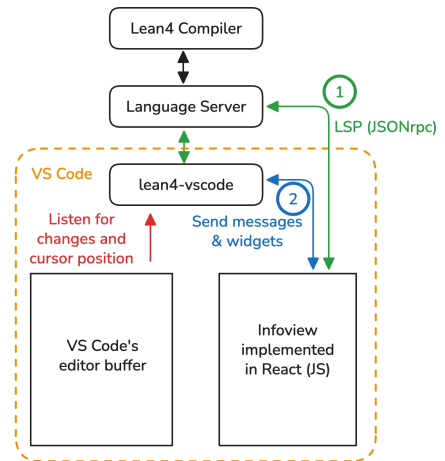


Figure 26: High-level architecture of Lean4’s user widget system in `lean4-vscode`.

The same architecture can be taken inspiration from for Avea and Agda, a proposal for an architecture can be found in Figure 27. The main component of this architecture is Iepje, an Elm-inspired web framework for Agda. Whereas widgets in Lean4 had to embed React code in the Lean source files, Avea would allow widgets to be written in Agda, maintaining all the aforementioned benefits of using one language.

The display buffer can be a new component implemented in Iepje, and its task will be to listen for new widgets and to handle all native messages from Avea (e.g. DisplayInfo and RunningInfo messages). The widgets themselves can also be implemented in Iepje.

1. If a widget should be loaded, Avea compiles the widget via the Agda handle.
2. Agda puts the compiled JS code in a temporary folder, which Avea communicates to the infoview.
3. The infoview can create a new DOM element for the new widget and loads the widget's compiled code from the temporary folder.
4. The widget is initialised with RPC controls by the infoview to have two-way communication with the extension, and is allowed to use the newly created DOM node to display.

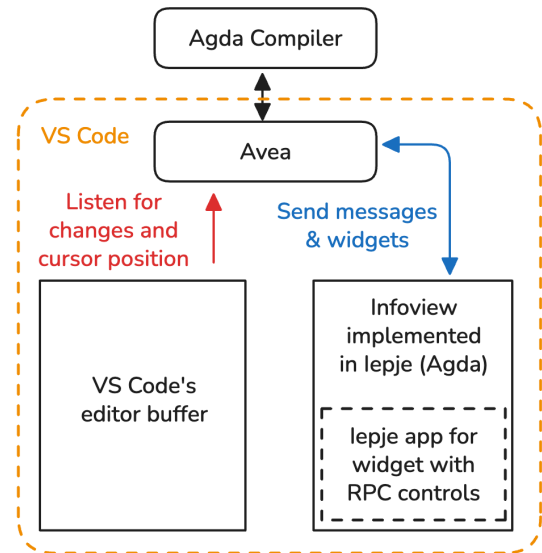


Figure 27: Architecture proposal for a version of Lean4's user widgets in Avea and Agda.

Use cases

There are numerous use cases for more complex visualisations in the display buffer:

- *Interactive dependency graphs*: A metaprogram can display an interactive dependency tree of a function to show information about the dependencies. This might include whether a dependency is a postulate without FFI definitions (useful for safety analysis).
- *References to symbols*: The results of a query on a program might return a list of symbols, or another expression. The display of symbols or expressions could be augmented by making them clickable, leading to the user to the definition of the symbols. For example, the safety analysis mentioned above could also list the postulates in the display buffer with clickable links to the declaration of the postulate.
- *Didactic purposes*: For example, an instructor could make an Agda file with coding exercises on graphs, and the graphs could be visualised in the display buffer to let students know what happens to the graphs while implementing graph operations.

6. Discussion

In this thesis, we have presented the functionality and implementation of a new Agda language extension for Visual Studio Code, called Avea. Avea is written in Agda, and is successful in functioning as well as other Agda language extensions. We have discussed what major challenges were found while developing a practical application in Agda compiling to Javascript, and discussed the solutions to them used in Avea.

Furthermore, we discussed how Avea can be made more extensible, drawing ideas from editor reflection proposed by Korkut and Christiansen [8]. However, given that Avea is written in Agda there are more opportunities to allow extensions to Avea to remain entirely written in Agda, while still allowing advanced tooling to be created.

While the proposed design for editor reflection in Agda can be used by all other Agda language extensions, given that it extends the interaction mode used by all of them. Avea can leverage its implementation in Agda to provide a comprehensible and type-safe API to call editor actions. This is an edge that Avea holds over all other language extensions.

Taking a broader view to other languages, we see that using editor reflection to make extensible language extensions need not be exclusive to Agda. Languages with a reflection API similar to Agda, such as Idris2¹⁰ and Rocq (using MetaRocq¹¹), are also candidates for editor reflection. Editor reflection could boost development in editor tooling in any of those languages too. We want to give special notice to Lean4, which has already implemented editor reflection to some extent: proof widgets are allowed to call meta-programs at edit time. However, proof widgets need to be written in Javascript, and the API to call meta-programs is not type-safe. Our work shows that this API could be made type-safe if proof widgets could be written in Lean too.

Lastly, we want to answer the two research questions posed at the start of the thesis:

1. Is Agda a suitable language to write practical programs with?

Yes, we believe that it is. Programs targeting Haskell can benefit from the relatively predictable compilation, which handles tail-call optimisation to do recursion over large data structures and imports. Targetting Javascript requires some additional infrastructure and programming disciplines: Javascript engines do not typically have tail-call optimisation, which means recursion should be used sparingly, and imports need to be handled outside of Agda currently. We think more development on the Javascript backend is beneficial since, Agda programs can tap into the vast ecosystem Javascript programs have access to.

2. Is it possible to make an extensible Agda extension written in Agda itself?

Even though this thesis does not provide an implementation of an *extensible* language extension for Agda, we still think it is possible. Leveraging Agda's meta-programming capabilities provides a better developer experience while keeping the extensibility API type-safe. Tooling developers need not learn Emacs Lisp (or another language of choice for the extension) to interface with the editor, and Haskell to write new tooling in Agda compiler's code base. Instead they can write Agda code to extend Avea, and use the reflection API to interface with the compiler. This extensibility API still allows for many different use cases, and has sufficient expressibility for advanced tools to be created.

¹⁰<https://github.com/stefan-hoeck/idris2-elab-util/blob/main/src/Doc/Index.md>

¹¹<https://metarocq.github.io/>

7. Future work

Besides implementing extensibility into Avea, there are other interesting research or engineering directions that can be explored that we have not discussed in this thesis.

Comparing runtime performance against other extensions

In the conclusion, the question “is Agda a suitable language to write practical programs with?” has been answered by arguing that it is possible to write Avea close to completely in Agda. During the development of Avea, we have not encountered many instances of performance bottlenecks due to Agda’s code generation, even though the current Javascript backend generates quite inefficient Javascript code. Further research could look into what Avea’s performance limits are, what other extensions’ performance limits are, what the performance limits of a hand-written extension in Javascript focussed on performance are, and what can be done in Agda’s Javascript backend to possibly improve the runtime performance of generated Javascript programs out-of-the-box.

Verifying parts of the extension

Writing Avea in Agda has proven to be useful in terms of allowing users to extend the editor tooling in Agda itself. However, with Agda’s ability as a theorem prover, it is also possible to verify parts of Avea’s functionality. While the formalisation of Avea is interesting of itself, the effects of the formalisation on the runtime performance are equally interesting.

Agda promotes the “correct by construction” style of proofs, where the proof of a property is usually intertwined with the function or algorithm the property applies to. This might incur costs on the runtime performance, and therefore, it might be interesting to look into how much the performance changes, and what might be done in the Agda compiler or Avea’s code to alleviate some of these performance costs.

Making the extension editor-agnostic

Looking at the implementations of existing Agda extensions, a lot of the infrastructure for handling user commands, and responses from the Agda process is quite similar (up to differences in programming language). Logically, there is no reason why Avea should only work in VS Code, since many of VS Code’s features Avea uses are also available in other editors. Therefore some engineering effort might be worth putting into making Avea editor-agnostic. Some options to do this are:

- To compile the Avea’s editor-agnostic part to WebAssembly, and let the extension for a new editor call functions from the WebAssembly module. The extension itself is then responsible for providing the bridge between Avea and the editor.
- Another possibility is to take inspiration from Rocq and Lean4, and to run Avea as a language server (with non-standard extensions for Agda interactions), with which a new editor extension can communicate with.

8. Bibliography

- [1] M. Dodds, ‘What Works (and Doesn’t) Selling Formal Methods’. Accessed: Mar. 13, 2026. [Online]. Available: <https://www.galois.com/articles/what-works-and-doesnt-selling-formal-methods>
- [2] S. Juhošová, A. Zaidman, and J. Cockx, ‘Pinpointing the learning obstacles of an interactive theorem prover’, in *2025 IEEE/ACM 33rd International Conference on Program Comprehension (ICPC)*, 2025, pp. 159–170.
- [3] S. Juhošová, A. Zaidman, and J. Cockx, ‘The Way of Types: A Report on Developer Experience with Type-Driven Development’, in *International Conference on Program Comprehension (ICPC)*. ACM. <https://doi.org/10.2026>.
- [4] R. Backx, ‘A Grounded Theory Study of "How Interactive Theorem Prover Programmers Write Code"’, Master's thesis, 2026.
- [5] Stack Overflow, ‘Stack Overflow Developer Survey 2025’. Accessed: Feb. 26, 2025. [Online]. Available: <https://survey.stackoverflow.co/2025/>
- [6] D. Christiansen and E. Brady, ‘Elaborator reflection: extending Idris in Idris’, in *Proceedings of the 21st acm sigplan international conference on functional programming*, 2016, pp. 284–297.
- [7] L. de Moura and S. Ullrich, ‘The Lean 4 Theorem Prover and Programming Language’, in *Automated Deduction – CADE 28: 28th International Conference on Automated Deduction, Virtual Event, July 12–15, 2021, Proceedings*, Berlin, Heidelberg: Springer-Verlag, 2021, pp. 625–635. doi: 10.1007/978-3-030-79876-5_37.
- [8] J. Korkut and D. T. Christiansen, ‘Extensible type-directed editing’, in *Proceedings of the 3rd ACM SIGPLAN International Workshop on Type-Driven Development*, in TyDe 2018. St. Louis, MO, USA: Association for Computing Machinery, 2018, pp. 38–50. doi: 10.1145/3240719.3241791.
- [9] W. Nawrocki, E. W. Ayers, and G. Ebner, ‘An Extensible User Interface for Lean 4’, in *14th International Conference on Interactive Theorem Proving (ITP 2023)*, A. Naumowicz and R. Thiemann, Eds, in Leibniz International Proceedings in Informatics (LIPIcs), vol. 268. Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2023, p. 24:1–24:20. doi: 10.4230/LIPIcs.ITP.2023.24.
- [10] P. D. van der Walt, ‘Reflection in agda’, Master's thesis, 2012.
- [11] W. Kokke and W. Swierstra, ‘Auto in agda: programming proof search using reflection’, in *International Conference on Mathematics of Program Construction*, 2015, pp. 276–301.
- [12] A. D. Barwell, C. Brown, M. S. Chang, C. Theoharis, and S. Thompson, ‘Structural Refactorings for Exploring Dependently Typed Programming’, in *Trends in Functional Programming: 25th International Symposium, TFP 2024, South Orange, NJ, USA, January 10–12, 2024, Revised Selected Papers*, South Orange, NJ, USA: Springer-Verlag, 2024, pp. 1–21. doi: 10.1007/978-3-031-74558-4_1.